

STAR Project

Comprehensive Gap Analysis & Implementation Roadmap

Organized by Codebase Directory

Date	2026-02-18
Branch	feat/gap-analysis-and-planning
Overall Completion	~75% (revised from 65%)
Firmware Assessment	Production-ready (all tasks complete)
Next Milestone	First Robot Motion (SPI bridge + UI telemetry)

Note on Accuracy: Initial analysis was based on an older codebase snapshot. Two items previously marked CRITICAL are confirmed complete:

Firmware WireMessage Dispatcher - **DONE** (comm_task.c:1959, 2060 lines)

BMS Telemetry Streaming - **DONE** (telemetry_task.c:1833, voltage + SOC)

STAR (Simultaneous Tracking and Robotics) – Locked Inc.

Contents

1	Executive Summary	3
1.1	System Architecture	3
1.2	Communication Flow	3
1.3	Overall Completion Matrix	3
2	Directory: e2-studio-star-rx72n-firmware/	5
2.1	What Is Complete	5
2.1.1	ThreadX Tasks (all 9 implemented)	5
2.1.2	Communication Stack (complete)	5
2.1.3	WireMessage Command Dispatcher (confirmed complete)	5
2.1.4	BMS Telemetry Streaming (confirmed complete)	6
2.1.5	Hardware Drivers (all complete)	7
2.2	What Is Missing	7
2.2.1	Gap F-1: OTA Firmware Update Handler (HIGH)	7
2.2.2	Gap F-2: NVS Configuration Persistence (HIGH)	8
2.2.3	Gap F-3: RPLiDAR C1 Integration (LOW)	9
2.3	Test Coverage	9
3	Directory: star-gateway/	10
3.1	Directory Structure	10
3.2	What Is Complete	10
3.2.1	Virtual RX72N Simulator (cmd/virtual_rx72n/)	10
3.3	What Is Missing	11
3.3.1	Gap G-1: FirmwareUpdateService - All 10 RPCs (HIGH)	11
3.3.2	Gap G-2: E-Stop Priority Queue (MEDIUM)	12
3.3.3	Gap G-3: SetMotorPower Not Dispatching to Firmware (MEDIUM)	12
3.3.4	Gap G-4: Virtual RX72N Motor Dynamics (LOW)	13
4	Directory: star-ros2/	14
4.1	Package Structure	14
4.2	What Is Complete	14
4.2.1	star_gateway_bridge Package (100%)	14
4.2.2	star_spi_bridge Package (90% - code complete)	14
4.3	What Is Missing	15
4.3.1	Gap R-1: SPI Bridge Hardware Testing (HIGH)	15
4.3.2	Gap R-2: star_bringup Launch Files (HIGH)	15
4.3.3	Gap R-3: Safety Monitor Test Coverage (MEDIUM)	16
4.3.4	Gap R-4: SLAM Stack (LOW - requires hardware first)	16
5	Directory: star-ui/	17
5.1	What Is Complete	17
5.2	What Is Missing	17
5.2.1	Gap U-1: gRPC-Web Bridge (CRITICAL - blocks all UI features)	17
5.2.2	Gap U-2: Emergency Stop Button (HIGH - safety critical)	17
5.2.3	Gap U-3: Telemetry Dashboard (HIGH)	18

5.2.4	Gap U-4: PID Configuration Panel (MEDIUM)	18
5.2.5	Gap U-5: Test Coverage (LOW)	19
6	Directory: star-proto/	20
6.1	Schema Inventory	20
6.2	nanopb Options Bug	20
6.3	Missing Wire Protocol Messages	20
7	Directory: .github/workflows/	22
7.1	Existing Workflows	22
7.2	Missing Workflows	22
7.2.1	Gap C-1: Firmware CI (firmware.yml) – HIGH	22
7.2.2	Gap C-2: UI CI (ui.yml) – MEDIUM	22
7.2.3	Gap C-3: Documentation CI (docs.yml) – LOW	22
8	Implementation Roadmap	24
8.1	Phase 1: Quick Wins (1-2 hours)	24
8.2	Phase 2: Enable First Robot Motion (2-4 days on hardware)	24
8.3	Phase 3: UI Telemetry (1-2 weeks)	24
8.4	Phase 4: NVS + OTA (1-2 weeks)	24
8.5	Phase 5: SLAM & Autonomy (months)	24
8.6	Effort Summary	25

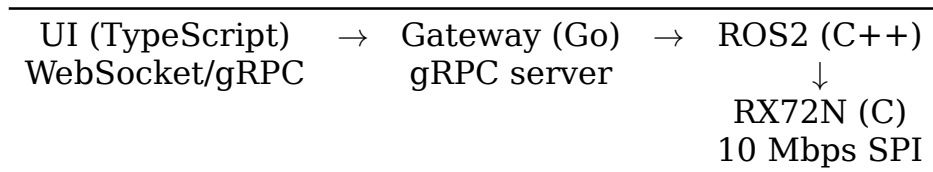
Executive Summary

STAR is a distributed robotics platform with a custom PCB RX72N motor controller, Raspberry Pi 5 control system, ROS2 Jazzy middleware, and a Go gateway service. The system communicates over 10 Mbps SPI using Protocol Buffers with HARQ/FEC error correction.

System Architecture

Directory	Language	Role
e2-studio-star-rx72n-firmware/	C23 (GNURX + ThreadX)	Motor controller firm
star-gateway/	Go	UI/ROS2 bridge, SPI
star-ros2/	C++ (ROS2 Jazzy)	Robot middleware st
star-ui/	TypeScript/React	Operator interface
star-proto/	Protocol Buffers 3	Message schemas
star-rpi5-buildroot/	Buildroot	Custom Linux for RP
matlab/	MATLAB	Motor system ID + F
schematic/	KiCad	PCB designs

Communication Flow



Overall Completion Matrix

Directory / Component	Completion	Status	Blocker
e2-studio-star-rx72n-firmware/			
Firmware core (9 ThreadX tasks)	100%	DONE	None
Communication stack (HARQ/FEC/CRC-32)	100%	DONE	None
WireMessage command dispatcher	100%	DONE	None
BMS telemetry streaming	100%	DONE	None
OTA firmware update handler	0%	MISSING	Firmwar
NVS configuration persistence	0%	MISSING	PID gain
RPLiDAR C1 driver	0%	MISSING	SLAM / r
star-gateway/			
Transport layer (SPI/USB CDC)	100%	DONE	None
Frame codec (HARQ/FEC/CRC-32)	100%	DONE	None
MotorControlService gRPC	80%	PARTIAL	E-Stop p
TelemetryService gRPC	90%	PARTIAL	GetSyste
BatteryManagementService gRPC	100%	DONE	None
ConfigurationService gRPC	85%	PARTIAL	SetRetra

Directory / Component	Completion	Status	Blocker
FirmwareUpdateService gRPC	0%	MISSING	All 10 R
Virtual RX72N simulator	95%	DONE	No moto
star-ros2/			
star_gateway_bridge_node	100%	DONE	None
star_spi_bridge_node (code)	90%	PARTIAL	Never ru
star_safety_monitor (tests)	40%	PARTIAL	3 of 10 t
star_bringup launch files	5%	MISSING	Empty sl
SLAM stack (RTAB-Map, EKF)	0%	MISSING	Requires
star-ui/			
Gamepad control	100%	DONE	None
gRPC-Web bridge	0%	MISSING	Required
Telemetry display (motors, sensors)	0%	MISSING	Operator
Battery status panel	0%	MISSING	None
Emergency Stop button	0%	MISSING	Safety cr
Configuration / PID panel	0%	MISSING	None
Tests (unit / E2E)	0%	MISSING	None
star-proto/			
Proto schemas (10 files, 73 messages)	100%	DONE	None
Generated code (Go/TypeScript/nanopb/C++)	100%	DONE	None
nanopb options (motor_status count)	85%	PARTIAL	max_cou
CI/CD (.github/workflows/)			
proto.yml (buf lint + generate)	100%	DONE	None
gateway.yml (Go build + test)	100%	DONE	None
ros2.yml (colcon build + test)	100%	DONE	None
firmware.yml (CMake build + test)	0%	MISSING	No firmw
ui.yml (npm build + type check)	0%	MISSING	No UI CI
docs.yml (LaTeX compile)	0%	MISSING	No docs

Directory: e2-studio-star-rx72n-firmware/

Language: C23 (GNURX toolchain) + ThreadX RTOS
Build: CMake + Renesas e² Studio
Hardware: Renesas RX72N (240 MHz, 4 MB Flash, 512 KB SRAM)
Overall: 84% production-ready. Core motor control, communication, and sensor stack is complete.

What Is Complete

ThreadX Tasks (all 9 implemented)

Task File	Purpose	Lines	Rate
src/tasks/app_main_task.c	RTOS entry, task creation	538	startup
src/tasks/comm_task.c	SPI/USB frame dispatch	2060	interrupt
src/tasks/motor_control_task.c	4-motor PID loop	2870	100 Hz
src/tasks/telemetry_task.c	Sensor aggregation + proto	1870	20 Hz
src/tasks/bms_monitor_task.c	Battery fault detection	1214	10 Hz
src/tasks/obstacle_detect_task.c	HC-SR04 sensor fusion	2202	50 Hz
src/tasks/temp_sensor_task.c	DS18B20 1-Wire polling	1227	1 Hz
src/tasks/led_status_task.c	Status LED patterns	411	20 Hz
src/tasks/watchdog_monitor_task.c	IWDT refresh + stack check	442	100 Hz

Communication Stack (complete)

Library	Lines	Capability
libs/rx_spi_comm/	2570	RSPI2 @ 10 Mbps, zero-copy buffers
libs/rx_usb_comm/	1575	USB CDC-ACM, 3 virtual COM ports
libs/rx_comm_manager/	2056	Multi-transport coordinator
libs/rx_fec/	869	Convolutional FEC, K=7, rate 1/2
libs/rx_harq/	675	Chase combining, configurable retries
libs/rx_frame/	895	SYNC+SEQ+LEN+TYPE+FLAGS+CRC-32
libs/rx_crc/	2923	CRC-32 (hardware) + CRC-8
libs/rx_session/	544	Cross-transport sequence continuity

WireMessage Command Dispatcher (confirmed complete)

src/tasks/comm_task.c:1959 - internal_handle_command_frame() handles all incoming protobuf commands.

```
static void internal_handle_command_frame(
    rx_comm_channel_t channel, const rx_frame_t* frame)
{
    /* Try SetVelocityRequest (4-motor differential drive) */
    err = rx_nanopb_decode_velocity_request(..., &velocity_req);
    if (err == k_rx_ok && velocity_req.has_command) {
```

```

    /* Map proto -> motor_command_t -> shared_data */
    cmd.target_velocity_mps[k_motor_idx_front_left] = ...;
    cmd.target_velocity_mps[k_motor_idx_front_right] = ...;
    cmd.target_velocity_mps[k_motor_idx_back_left] = ...;
    cmd.target_velocity_mps[k_motor_idx_back_right] = ...;
    shared_data_set_motor_command(&cmd);    // -> wakes
motor_control_task
    return;
}

/* Try EmergencyStopRequest */
err = rx_nanopb_decode_estop_request(..., &estop_req);
if (err == k_rx_ok) {
    shared_data_trigger_estop(k_estop_reason_manual); // -> POEG HW
stop
    return;
}

/* Try SetPIDGainsRequest */
err = rx_nanopb_decode_pid_gains_request(..., &pid_req);
if (err == k_rx_ok && pid_req.has_pid_config) {
    gains.kp = (float)pid_req.pid_config.kp; // Kp, Ki, Kd
    shared_data_set_pid_gains(&gains);
    return;
}

/* Try SetRetransmitConfigRequest */
err = rx_nanopb_decode_retransmit_config_request(..., &retransmit_req);
if (err == k_rx_ok && retransmit_req.has_retransmit_config) {
    rx_comm_manager_set_auto_retransmit(...);
    return;
}

rx_log_warn(s_tag, "Could not decode command payload");
}

```

Listing 1: Command dispatcher in comm_task.c (lines 1959–2060)

BMS Telemetry Streaming (confirmed complete)

src/tasks/telemetry_task.c:1833 – BMS data is included in every TelemetryData protobuf frame sent to the gateway.

```

if (err == k_rx_ok && bms_state.valid) {
    /* Convert millivolts to volts for protobuf field */
    telemetry.battery_voltage_v = (float)bms_state.voltage_mv
                                / s_mv_per_volt;
    telemetry.battery_soc_percent = bms_state.soc_percent;
    telemetry.battery_percent     = (float)bms_state.soc_percent;
}

```

Listing 2: BMS integration in telemetry_task.c (lines 1833–1838)

Hardware Drivers (all complete)

Driver	Lines	Hardware
libs/rx_motor/	1936	DRV8243S H-bridge, PWM dead-time
libs/rx_pid/	1582	Discrete-time PID, anti-windup
libs/rx_encoder/	1969	MTU1/2 + TPU quadrature encoders
libs/rx_drv8243/	1983	DRV8243S SPI config driver
libs/rx_bq4050/	1293	BQ4050 BMS via SMBus (RIIC1)
libs/rx_ds18b20/	1429	DS18B20 1-Wire temperature
libs/rx_hcsr04/	4761	HC-SR04 ultrasonic, IRQ-based
libs/rx_hal/	16043	Full RX72N HAL (GPIO/SPI/UART/ADC)
libs/rx_bus/	11941	Bus manager (I2C/SPI/1-Wire/UART)

What Is Missing

Gap F-1: OTA Firmware Update Handler (HIGH)

Impact: Cannot update firmware in production without re-flashing over JTAG. All 10 FirmwareUpdateService RPCs in the gateway return Unimplemented. Neither the firmware nor the gateway has implemented this.

Files to create:

```
e2-studio-star-rx72n-firmware/
  libs/rx_ota/
    inc/rx_ota.h      -- State machine + flash write API
    src/rx_ota.c      -- Implementation
  tests/
    test_rx_ota.c     -- Unity tests with mock flash
```

Listing 3: New files needed for OTA

OTA state machine:

```
typedef enum : uint8_t {
    k_ota_state_idle      = 0,  /**< No update in progress */
    k_ota_state_starting  = 1,  /**< BeginUpdate received */
    k_ota_state_receiving = 2,  /**< WriteChunk frames arriving */
    k_ota_state_validating = 3, /**< Checking CRC-32 + size */
    k_ota_state_ready     = 4,  /**< Ready to reboot */
    k_ota_state_error     = 5,  /**< Error state */
} ota_state_t;

typedef struct {
    ota_state_t state;           /**< Current FSM state */
    uint32_t total_size;         /**< Expected firmware size (bytes) */
    uint32_t bytes_written;      /**< Progress counter */
    uint32_t expected_crc;       /**< Expected CRC-32 from BeginUpdate */
    uint32_t running_crc;        /**< Running CRC accumulator */
    uint32_t session_id;         /**< From BeginUpdateRequest.header */
} ota_context_t;

rx_err_t rx_ota_begin(const star_v1_BeginUpdateRequest* req);
rx_err_t rx_ota_write_chunk(const star_v1_FirmwareChunk* chunk);
```

```
rx_err_t rx_ota_finalize(void);
rx_err_t rx_ota_abort(void);
rx_err_t rx_ota_get_progress(uint8_t* percent_out);
rx_err_t rx_ota_reboot(void);
```

Listing 4: OTA state machine types (rx_ota.h)

Note: The wire.proto WireMessage oneof needs a FirmwareChunk field added before OTA transfer can work. Also add the OTA message types to internal_handle_con

Estimated effort: 2–3 days (firmware state machine + flash write + gateway integration)

Gap F-2: NVS Configuration Persistence (HIGH)

Impact: PID gains received via SetPIDGainsRequest are applied to the running system via shared_data_set_pid_gains() but are NOT saved to flash. Every reboot resets to compile-time defaults.

The RX72N has 64 KB Data Flash (separate from code flash) ideal for NVS. No library currently uses it.

Files to create:

```
e2-studio-star-rx72n-firmware/
  libs/rx_nvs/
    inc/rx_nvs.h    -- Key-value store API
    src/rx_nvs.c    -- Data Flash implementation
  tests/
    test_rx_nvs.c   -- Unity tests (mock flash)
```

Listing 5: NVS library structure

```
/**
 * @brief Write named config blob to Data Flash.
 * @param[in] key  ASCII key (max 32 chars)
 * @param[in] data  Buffer to save
 * @param[in] len   Length (max 4096 bytes)
 */
rx_err_t rx_nvs_write(const char* key,
                     const void* data, uint16_t len);

/**
 * @brief Read named config blob from Data Flash.
 */
rx_err_t rx_nvs_read(const char* key,
                    void* data, uint16_t* len);

/** @brief Factory reset (erase all Data Flash). */
rx_err_t rx_nvs_erase_all(void);
```

Listing 6: NVS API (rx_nvs.h)

Integration points after creating NVS:

- `src/tasks/comm_task.c` – After `shared_data_set_pid_gains()`, call `rx_nvs_write("pid_config", ...)`

- `src/main.c` – On startup, call `rx_nvs_read("pid_config", ...)` before `tx_kernel_enter()`

Estimated effort: 4–6 hours

Gap F-3: RPLiDAR C1 Integration (LOW)

Impact: SLAM / autonomous navigation is impossible without the lidar sensor. RPLiDAR C1 (12 m range, 360°, IP54) is listed in the hardware design but has no firmware driver.

Interface: UART at 460800 baud. Requires a dedicated SCI channel (SCI1 or SCI2 recommended; SCI9 is the debug console).

Files to create:

```
e2-studio-star-rx72n-firmware/
  libs/rx_rplidar/
    inc/rx_rplidar.h
    src/rx_rplidar.c
  src/tasks/
    lidar_scan_task.c  -- 10 Hz scan polling
  tests/
    test_rx_rplidar.c
```

Listing 7: RPLiDAR library structure

Proto change needed (add to `telemetry.proto` or new `lidar.proto`):

```
message LidarScan {
  uint64 timestamp_us = 1;
  repeated LidarPoint points = 2;  // ~360 points per scan

  message LidarPoint {
    float angle_deg = 1;  // 0-360 degrees
    float distance_mm = 2;  // 0 = invalid measurement
    uint32 quality = 3;  // Signal quality 0-255
  }
}
```

Listing 8: Required proto additions for LiDAR

Estimated effort: 2–3 days (driver + task + proto + nanopb options)

Test Coverage

The firmware has 87 test files covering all implemented modules. Test infrastructure uses Unity + CMock with mock hardware injection via function-pointer bus interfaces (SOLID Dependency Inversion).

```
tests/
  test_rx_ota.c          -- OTA state machine + CRC validation
  test_rx_nvs.c          -- Data Flash read/write/erase
  test_rx_rplidar.c      -- UART protocol parsing (mock UART)
```

Listing 9: Test files for gap areas (to be created)

Directory: star-gateway/

Language: Go 1.22
Build: go build ./cmd/star-gateway
Role: Bridges gRPC clients (UI, ROS2) to the RX72N via SPI/USB CDC
Test coverage: 80.8% (runs in CI via .github/workflows/gateway.yml)
Overall: Transport, framing, FEC, and HARQ are production-ready. FirmwareUpdateService is 0%.

Directory Structure

```

star-gateway/
cmd/
  star-gateway/main.go      -- Entry point
  virtual_rx72n/           -- Simulator (95% complete)
internal/
  transport/               -- Layer 1: SPI + USB CDC
  frame/                   -- Layer 2: SYNC+SEQ+LEN+TYPE+FLAGS+CRC-32
  fec/                     -- FEC: Viterbi K=7, rate 1/2
  harq/                    -- HARQ: Chase Combining retransmit
  service/                 -- gRPC service handlers
    motor_control.go       -- MotorControlService (80%)
    telemetry.go           -- TelemetryService (90%)
    battery.go             -- BatteryManagementService (100%)
    configuration.go       -- ConfigurationService (85%)
    firmware.go            -- FirmwareUpdateService (0% - all stubs)
    gateway_service.go     -- GatewayService for ROS2 bridge (85%)
  manager/                 -- Transport manager, dispatcher
  link/                    -- SPI link layer
  controller/              -- Gamepad state

```

Listing 10: star-gateway/ layout

What Is Complete

Package	Coverage	Status
internal/frame	100%	DONE
internal/fec	100%	DONE
internal/harq	87%	DONE
internal/transport	92.1%	DONE
internal/manager	97.4%	DONE
internal/service	94.4%	PARTIAL (firmware.go is 0%)
internal/controller	100%	DONE
internal/link	83.3%	DONE

Virtual RX72N Simulator (cmd/virtual_rx72n/)

The simulator is a Go TCP/Unix socket server that replaces real hardware for integration testing.

Feature	Status
Full SPI frame protocol (SYNC+SEQ+LEN+TYPE+FLAGS+CRC-32)	DONE
PING/PONG heartbeat with counter echo	DONE
RESET/RESET_ACK session synchronization	DONE
Protobuf VelocityCommand and EmergencyStop handling	DONE
Simulated TelemetryData (static values)	DONE
Sequence number tracking with wraparound at 0xFFFF	DONE
ACK frame generation for FlagRequiresAck frames	DONE
Configurable latency injection (SIM_LATENCY_MS env var)	DONE
Graceful shutdown with socket cleanup	DONE
Motor dynamics simulation ($G(s) = 3.665/(0.075s+1)$)	MISSING
Fault injection API (CRC errors, packet loss)	MISSING
BMS state of charge simulation	MISSING

Motor dynamics model (to implement in `cmd/virtual_rx72n/motor_sim.go`):

```
// First-order model: G(s) = K/(tau*s + 1)
// K = 3.665 rad/s/V, tau = 0.075 s
// Discrete backward Euler at T=0.010 s:
//   v[k] = (1 - exp(-T/tau)) * K * u[k] + exp(-T/tau) * v[k-1]
//         = 0.1248 * K * u[k] + 0.8752 * v[k-1]
alpha := math.Exp(-motorSamplePeriod / motorTimeConstant) // 0.8752
m.velocityRadps = (1-alpha)*motorGain*inputVoltage +
    alpha*m.velocityRadps
m.positionRad += m.velocityRadps * motorSamplePeriod
ticks := int64(m.positionRad / (2*math.Pi) * ticksPerRev)
m.encoderTicks.Store(ticks)
```

Listing 11: Motor dynamics (discrete-time, 100 Hz)

What Is Missing

Gap G-1: FirmwareUpdateService - All 10 RPCs (HIGH)

File: `internal/service/firmware.go`

Impact: Production firmware updates are completely blocked. No OTA capability exists.

Every method returns: `status.Error(codes.Unimplemented, "not implemented")`

RPC Method	Status
BeginUpdate	MISSING (TODO: Initialize OTA partition, set expected size)
WriteChunk	MISSING (TODO: Write single firmware chunk)
StreamChunks	MISSING (TODO: Bulk chunk transfer via client streaming)
FinalizeUpdate	MISSING (TODO: Validate firmware, prepare reboot)
AbortUpdate	MISSING (TODO: Cancel and cleanup)
GetUpdateProgress	MISSING (TODO: Return bytes received / total)
StreamUpdateProgress	MISSING (TODO: Server-stream progress events)
Reboot	MISSING (TODO: Trigger reboot into new firmware)
Rollback	MISSING (TODO: Boot previous firmware)
MarkValid	MISSING (TODO: Confirm new firmware stable)

Required implementation:

1. OTA session state machine (idle → starting → receiving → validating → ready)
2. Chunk buffer with running CRC-32 verification
3. HARQ-reliable chunk transmission (each chunk requires ACK from firmware)
4. Firmware image validation (size + CRC match from BeginUpdate)
5. Partition switching command to firmware
6. Reboot sequencing with health check timeout
7. Rollback on failed health check

Estimated effort: 2-3 days (gateway side only; firmware OTA handler also required)

Gap G-2: E-Stop Priority Queue (MEDIUM)

File: `internal/service/motor_control.go:90`

TODO comment: “Implement priority framing for E-Stop.”

Impact: Emergency stop goes through normal HARQ queue. During link congestion, the stop command could be delayed behind pending telemetry ACKs.

Required: A priority transmission path that bypasses the normal frame queue and sends the E-Stop frame immediately, clearing any pending outgoing frames.

Estimated effort: 2-3 hours

Gap G-3: SetMotorPower Not Dispatching to Firmware (MEDIUM)

The SetMotorPower RPC in MotorControlService accepts MotorPowerCommand but does not send it over the wire to the RX72N. The firmware command dispatcher does not handle MotorPowerCommand either (only SetVelocityRequest is handled).

Required:

1. Add MotorPowerCommand to the WireMessage oneof in wire.proto
2. Add handler in `e2-studio-star-rx72n-firmware/src/tasks/comm_task.c`
3. Dispatch from SetMotorPower RPC handler in gateway

Estimated effort: 3-4 hours (proto change + firmware + gateway)

Gap G-4: Virtual RX72N Motor Dynamics (LOW)

The simulator returns static telemetry regardless of commanded velocity. Motor dynamics simulation would enable:

- Testing odometry and dead-reckoning algorithms
- PID controller validation in simulation before hardware
- Realistic encoder tick responses to velocity commands

File to create: `cmd/virtual_rx72n/motor_sim.go` (see motor model above)

Estimated effort: 1-2 days (motor dynamics + BMS simulation + fault injection)

Directory: star-ros2/

Language: C++ (ROS2 Jazzy, C++17)
Build: colcon build + clang-format enforcement
Role: ROS2 middleware – bridges gateway gRPC to robot topics, manages safety
Overall: Core bridge nodes are complete. Hardware integration untested. SLAM stack not started.

Package Structure

```
star-ros2/
  star_gateway_bridge/      -- gRPC client -> ROS2 topics (COMPLETE)
    src/
      star_gateway_bridge_node.cpp  -- ForwardTelemetry, GetTeleopCommand
      , SetPIDGains
      message_converter.cpp         -- Proto <-> ROS2 message translation
  star_spi_bridge/          -- SPI driver ROS2 node (code complete,
    untested)
    src/
      star_spi_driver_node.cpp       -- SPI framing + HARQ for ROS2
  star_safety_monitor/      -- Safety watchdog (40% test coverage)
    src/
      safety_monitor_node.cpp        -- Battery/timeout/velocity safety
    checks
  star_bringup/             -- Launch files (EMPTY SKELETON)
    launch/                  -- No launch files exist yet
  star_nav/                 -- Navigation stack (0% - not started)
```

Listing 12: star-ros2/ package layout

What Is Complete

star_gateway_bridge Package (100%)

Fully implemented. Bridges between gateway gRPC service and ROS2 topics/services.

Function	ROS2 Interface	gRPC Call
Teleop forwarding	/velocity_command (sub)	GetTeleopCommand()
Telemetry bridge	/robot_status (pub)	ForwardTelemetry()
PID gain update	/pid_gains (sub)	SetPIDGains()

star_spi_bridge Package (90% - code complete)

All code is written but has **never been run on real hardware** (RPi5 + RX72N). This is the final integration point before the robot can physically move.

Known issue in `src/star_spi_driver_node.cpp:112`:

```
// CURRENT (unsafe):
CallbackReturn on_deactivate(const State & previous_state) override {
    transport_->close();    // Closes SPI -- but motors may still be
                           // spinning!
    return CallbackReturn::SUCCESS;
}

// REQUIRED (safe):
CallbackReturn on_deactivate(const State & previous_state) override {
    // Send zero-velocity frame before closing SPI
    auto zero_cmd = build_velocity_command(0.0f, 0.0f, 0.0f, 0.0f);
    transport_->send(zero_cmd);
    std::this_thread::sleep_for(std::chrono::milliseconds(50));
    transport_->close();
    return CallbackReturn::SUCCESS;
}
```

Listing 13: Safety gap: on_deactivate() does not send zero velocity

What Is Missing

Gap R-1: SPI Bridge Hardware Testing (HIGH)

Effort: 4-8 hours on hardware

Required: RPi5 with /dev/spidev0.0 connected to RX72N running firmware.

Validates: Frame encoding, HARQ retransmit, telemetry reception, encoder feedback.

Checklist before hardware test:

- Fix on_deactivate() zero-velocity safety (see above)
- Verify SPI CS polarity matches firmware (active-low)
- Confirm 10 MHz SPI mode 0 configured in RPi5 device tree
- Verify COPI/CIPO pin assignments match firmware pinout

Gap R-2: star_bringup Launch Files (HIGH)

Directory: star_bringup/launch/ – **completely empty**

Impact: Cannot start the full robot system. Every node must be launched manually.

Required launch files:

```
star_bringup/launch/
star_robot.launch.py    -- Full system: bridge + SPI + safety + nav
star_simulation.launch.py -- Gazebo + virtual_rx72n simulator
star_teleop.launch.py   -- Manual control only (bridge + SPI + safety)
star_nav.launch.py      -- Autonomous: adds SLAM + NavStack
```

Listing 14: Required launch files for star_bringup

Estimated effort: 4-6 hours

Gap R-3: Safety Monitor Test Coverage (MEDIUM)

`star_safety_monitor/` has 40% test coverage – 3 of 10 test cases use `GTEST_SKIP`.

Battery safety checks and E-Stop triggering paths are not tested.

Tests that need implementation:

- Battery critically low → E-Stop triggers (currently skipped)
- Over-temperature → velocity limit applied (currently skipped)
- Communication timeout → E-Stop triggers (currently skipped)

Gap R-4: SLAM Stack (LOW - requires hardware first)

Component	Status
RTAB-Map (mapping + localization)	Not installed or configured
robot_localization EKF	Not configured
Nav2 (path planning + obstacle avoid)	Not installed or configured
RPLiDAR C1 ROS2 driver	Not integrated
IMU (MPU-6050 or ICM-42688)	Hardware not yet chosen

Prerequisite: RPLiDAR C1 firmware driver (Gap F-3) must be completed first.

Directory: `star-ui/`

Language: TypeScript / React

Build: npm / Vite

Role: Operator interface – gamepad control + robot state visualization

Overall: Gamepad control works. Everything else is missing. 0% test coverage.

What Is Complete

Gamepad control – WebSocket connection to gateway, gamepad input captured and forwarded as teleop commands. This is the only working UI feature.

What Is Missing

Gap U-1: gRPC-Web Bridge (CRITICAL – blocks all UI features)

Impact: The UI cannot call any gRPC service (telemetry, battery, motor control, config, OTA). A gRPC-Web proxy is required between the browser and the Go gateway gRPC server.

Options:

1. **Envoy proxy** (recommended for production): Add `envoy.yaml` configuration, run as sidecar on RPi5
2. **grpc-gateway** (simpler): Add REST transcoding to gateway, serves HTTP/JSON that browsers can call
3. **connect-go** (modern): Replace `grpc-go` with `connect-go`, which speaks HTTP/1.1 natively

Recommended approach (grpc-gateway transcoding, ~4 hours):

```
// In cmd/star-gateway/main.go, add HTTP gateway:
mux := runtime.NewServeMux()
opts := []grpc.DialOption{grpc.WithTransportCredentials(
    insecure.NewCredentials())}
starv1.RegisterMotorControlServiceHandlerFromEndpoint(
    ctx, mux, "localhost:50051", opts)
starv1.RegisterTelemetryServiceHandlerFromEndpoint(
    ctx, mux, "localhost:50051", opts)
// etc. for all 5 services
http.ListenAndServe(":8080", mux)
```

Listing 15: Gateway REST transcoding setup

Estimated effort: 4–8 hours (bridge setup + proto HTTP option annotations)

Gap U-2: Emergency Stop Button (HIGH – safety critical)

Impact: Operators cannot issue emergency stop from the UI. The only E-Stop is via gamepad button – no visible on-screen control exists.

Required:

- Prominent red EMERGENCY STOP button (always visible, not behind navigation)
- Calls POST /v1/motor/emergency_stop (via gRPC-Web bridge)
- Keyboard shortcut: Space or Escape
- State display: shows ACTIVE / CLEARED status

Estimated effort: 2–3 hours (after gRPC-Web bridge is working)

Gap U-3: Telemetry Dashboard (HIGH)

Impact: Operators are completely blind to robot state. Without telemetry display, the robot cannot be safely operated even with gamepad control.

Required components:

UI Component	Data Source	Update Rate
Motor velocity (4x)	StreamEncoders	100 Hz
Encoder ticks (4x)	StreamEncoders	100 Hz
Battery voltage (V)	StreamBatteryState	1 Hz
State of charge (%)	StreamBatteryState	1 Hz
System uptime	GetSystemStatus	1 Hz
Transport health	TransportDiagnostics	1 Hz
Obstacle distances (4x)	StreamTelemetry	20 Hz
Temperature (°C)	StreamTelemetry	1 Hz

Estimated effort: 12–20 hours (React components + gRPC streaming)

Gap U-4: PID Configuration Panel (MEDIUM)

Allows operators to tune PID gains live and apply to the running firmware via SetMotorPidConfig RPC.

Required:

- Input fields for Kp, Ki, Kd per motor (or global)
- Send via POST /v1/config/motor_pid
- Display current gains (from GET /v1/config/motor_pid)
- Visual feedback: success/failure confirmation

Estimated effort: 4–6 hours

Gap U-5: Test Coverage (LOW)

Currently 0% test coverage in `star-ui/`. No unit, integration, or E2E tests exist.

Recommended framework: Vitest + React Testing Library for unit tests; Playwright for E2E.

Directory: star-proto/

Language: Protocol Buffers 3 (buf toolchain)
Targets: Go, TypeScript, nanopb C, C++ (gRPC)
Overall: Schema definitions are complete. One known nanopb options bug. Generated code is checked in.

Schema Inventory

Proto File	Messages	Services	St
proto/star/v1/common.proto	5	-	DC
proto/star/v1/motor_control.proto	9	1 (5 RPC)	DC
proto/star/v1/telemetry.proto	7	1 (3 RPC)	DC
proto/star/v1/battery_management.proto	15	1 (10 RPC)	DC
proto/star/v1/configuration.proto	13	1 (7 RPC)	DC
proto/star/v1/firmware_update.proto	12	1 (10 RPC)	DC
proto/star/v1/gateway_service.proto	6	1 (3 RPC)	DC
proto/star/v1/diagnostics.proto	2	-	DC
proto/star/v1/controller.proto	1	-	DC
proto/star/v1/wire.proto	2	-	DC
Total	72	5 services / 38 RPC	

nanopb Options Bug

File: nanopb/star/v1/motor_control.options
Bug: star.v1.SetVelocityResponse.motor_status max_count:2
Should be: max_count:4 (the robot has 4 motors: FL, FR, BL, BR)
Impact: Response messages silently truncate motor 3 and motor 4 status. Only motors 0 and 1 are readable in responses.

Fix (30 minutes):

```
# CURRENT (WRONG):
star.v1.SetVelocityResponse.motor_status max_count:2

# CORRECT:
star.v1.SetVelocityResponse.motor_status max_count:4
```

Listing 16: motor_control.options fix

After fixing: run `make proto-gen` && `make proto-gen-firmware` to regenerate all targets.

Missing Wire Protocol Messages

For OTA firmware updates to work, `wire.proto` needs additions to the `WireMessage` oneof:

```
// Current WireMessage oneof handles:  
//   VelocityCommand, EmergencyStopCommand, MotorPowerCommand,  
//   TelemetryData, EncoderData, BatteryStatus, PidConfig,  
//   RetransmitConfig, TransportDiagnostics  
  
// MISSING (needed for OTA):  
oneof payload {  
    // ... existing fields ...  
    FirmwareChunk firmware_chunk = 60; // OTA chunk transfer  
    BeginUpdateRequest begin_update = 61;  
    FinalizeUpdateRequest finalize_update = 62;  
}
```

Listing 17: Required additions to wire.proto WireMessage

Directory: `.github/workflows/`

Existing CI: proto.yml, gateway.yml, ros2.yml (all working)
Missing CI: firmware build, UI build, documentation compile
Impact: Firmware and UI regressions can merge undetected.

Existing Workflows

Workflow	Triggers	Steps
proto.yml	Push to star-proto/	buf lint, breaking check, generate, test
gateway.yml	Push to star-gateway/	build, test (-race), lint, security, cross-compile
ros2.yml	Push to star-ros2/	colcon build, ament_lint, unit tests

Missing Workflows

Gap C-1: Firmware CI (firmware.yml) - HIGH

No CI exists for the RX72N firmware. Build and test failures only discovered locally.

```
# .github/workflows/firmware.yml
steps:
  - Install GNURX toolchain (rxelf-unknown-elf-gcc)
  - cmake -B build -DCMAKE_TOOLCHAIN_FILE=cmake/rx72n.cmake
  - cmake --build build --target all # Firmware binary
  - cmake --build build --target run_tests # Unity tests via CTest
  - Upload test results as artifact
  - Check for Doxygen warnings
```

Listing 18: firmware.yml required steps

Gap C-2: UI CI (ui.yml) - MEDIUM

```
# .github/workflows/ui.yml
steps:
  - npm ci
  - npm run type-check # TypeScript strict mode
  - npm run lint # ESLint
  - npm run test # Vitest unit tests
  - npm run build # Production build (catches tree-shaking issues)
```

Listing 19: ui.yml required steps

Gap C-3: Documentation CI (docs.yml) - LOW

```
# .github/workflows/docs.yml
steps:
  - apt-get install texlive-xetex texlive-fonts-extra
  - cd docs && make # Compile star_documentation.pdf
  - cd .worktrees/planning/plans && xelatex STAR_Gap_Analysis.tex
  - Check for LaTeX errors (exit code)
```

- Upload PDFs as artifacts

Listing 20: docs.yml required steps

Implementation Roadmap

Phase 1: Quick Wins (1-2 hours)

1. **Fix nanopb motor_status bug** (`star-proto/nanopb/star/v1/motor_control.o`)
Change `max_count:2` → `max_count:4`. Run `make proto-gen`.
2. **Fix SPI bridge zero-velocity safety** (`star-ros2/star_spi_bridge/src/star_spi`)
Send zero-velocity `WireMessage` before closing SPI in `on_deactivate()`.

Phase 2: Enable First Robot Motion (2-4 days on hardware)

3. **Hardware bring-up** – Verify SPI wiring (COPI/CIPO/CS/CLK) matches firmware pinout
4. **Run SPI bridge on hardware** – Exercise motion path end-to-end with `virtual_rx72n` first, then real firmware
5. **Create star_bringup launch files** – `star_robot.launch.py` for full system startup
6. **Verify teleop path:** UI gamepad → Gateway → ROS2 → SPI bridge → RX72N motor spin

Phase 3: UI Telemetry (1-2 weeks)

7. **gRPC-Web bridge** – `grpc-gateway` or `Envoy` for browser access to gateway gRPC services
8. **Emergency Stop button** – Safety-critical UI component
9. **Telemetry dashboard** – Motor velocities, encoders, battery, temperature, obstacle distances
10. **PID configuration panel** – Live gain tuning with NVS persistence

Phase 4: NVS + OTA (1-2 weeks)

11. **NVS library** (`e2-studio-star-rx72n-firmware/libs/rx_nvs/`) – Data Flash key-value store
12. **PID persistence** – Save/restore gains across reboot
13. **OTA firmware handler** (`libs/rx_ota/`) – State machine + Data Flash write
14. **FirmwareUpdateService** (`star-gateway/internal/service/firmware.go`)
– Implement all 10 stubs

Phase 5: SLAM & Autonomy (months)

15. RPLiDAR C1 firmware driver (`libs/rx_rplidar/`)

16. RTAB-Map configuration for ROS2 Jazzy
17. robot_localization EKF with encoder odometry
18. Nav2 path planner configuration
19. System integration testing

Effort Summary

Gap	Severity	Effort
Fix nanopb max_count:4	Critical bug	30 min
Fix SPI on_deactivate() zero-vel	Safety	1 hr
star_bringup launch files	High	4-6 hrs
SPI bridge hardware testing	High	4-8 hrs
gRPC-Web bridge	High	4-8 hrs
Emergency Stop UI button	High (safety)	2-3 hrs
Telemetry dashboard	High	12-20 hrs
Safety monitor test coverage	Medium	4-6 hrs
NVS library (rx_nvs)	High	4-6 hrs
PID gain persistence	High	2 hrs
E-Stop priority queue (gateway)	Medium	2-3 hrs
SetMotorPower firmware dispatch	Medium	3-4 hrs
Firmware CI (firmware.yml)	Medium	4-6 hrs
UI CI (ui.yml)	Medium	2-3 hrs
OTA handler (firmware)	High	2-3 days
FirmwareUpdateService (gateway)	High	2-3 days
PID config UI panel	Medium	4-6 hrs
Virtual RX72N motor dynamics	Low	1-2 days
Virtual RX72N fault injection	Low	4-8 hrs
RPLiDAR C1 driver	Low	2-3 days
SLAM stack	Low	weeks